

COMP3069 Computer Graphics

Lab 6

Originally prepared by Dr Kristian Spoerer of UNUK
Modified by Prof Sean He of UNNC

Learning Outcomes

After Lab 6, you are expected to be able to

1. Use a model, view and projection matrix by writing OpenGL and corresponding the GLSL code, to translate, rotate and scale 3-Dimensional points.
2. Demonstrate that you can apply transformations to models and a camera to arrange objects in 3D and render a scene.
3. Specify and use a view matrix by writing OpenGL and corresponding the GLSL code, to translate and rotate a camera. Apply transformations to the camera to setup and render a scene.
4. Interpret the position and orientation of models and a camera from an illustration of the scene.
5. Interpret different views of the same 3D scene, (front, top, side and perspective views).
6. Compose sequences of 3D transformations and apply them to single and multiple objects.
7. Infer the correct order of a sequence of 3D transformations to achieve the desired effect.
8. Recognise the difference between clockwise and anti-clockwise rotation.
9. Recognise that the same effect can be achieved using sequences of different transformations.
10. Diagnose possible incorrect transformations that result in an undesirable rendered image.

Part A Set up

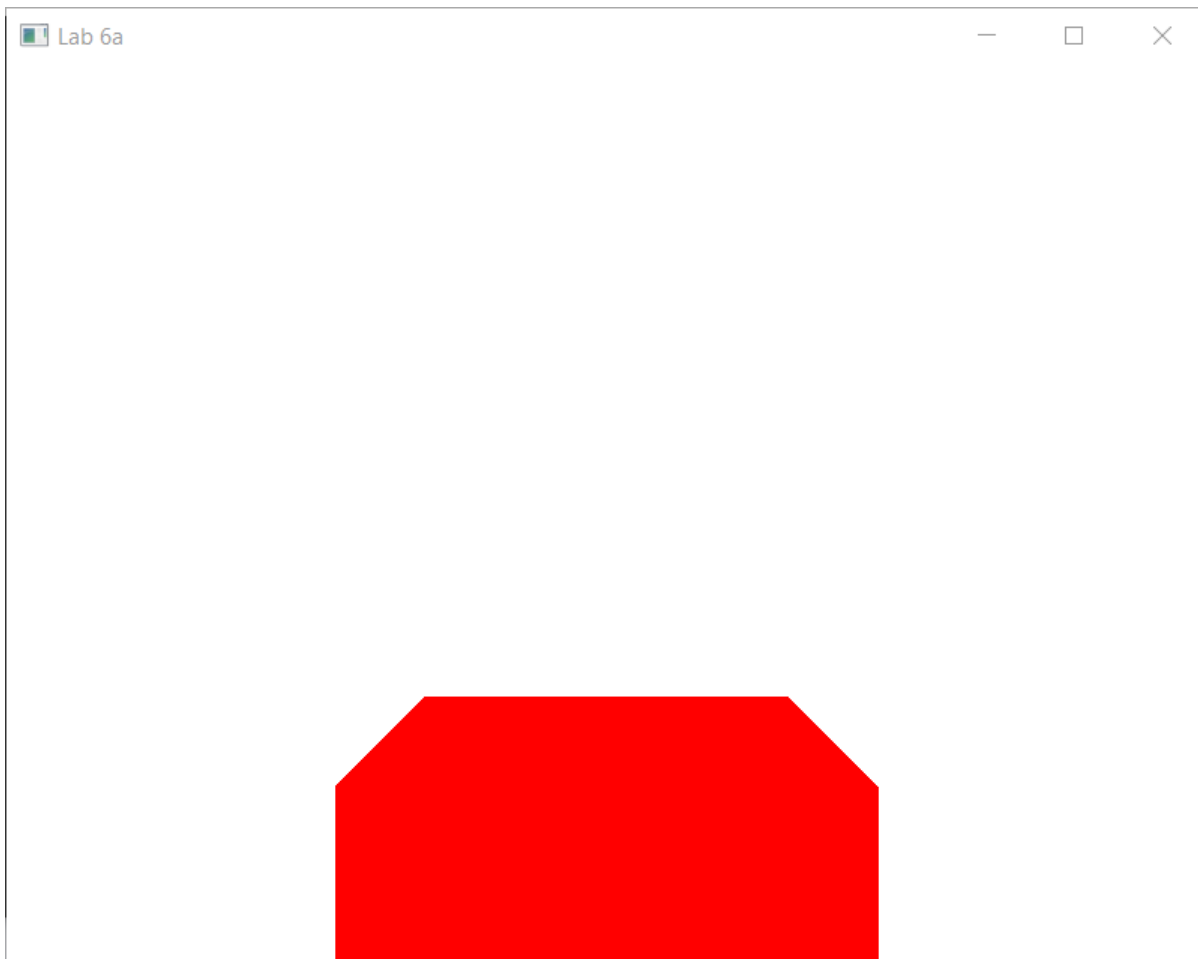
In the COMP3069 directory, create a new project called

Lab6a

Download Lab6a.cpp from Moodle and copy the code to the Lab6a.cpp in your new project.

Download the vertex shader mvp.vert and fragment shader uniform.frag from Moodle, copy them to your project, and add them to the Source Files in the Solution Explorer.

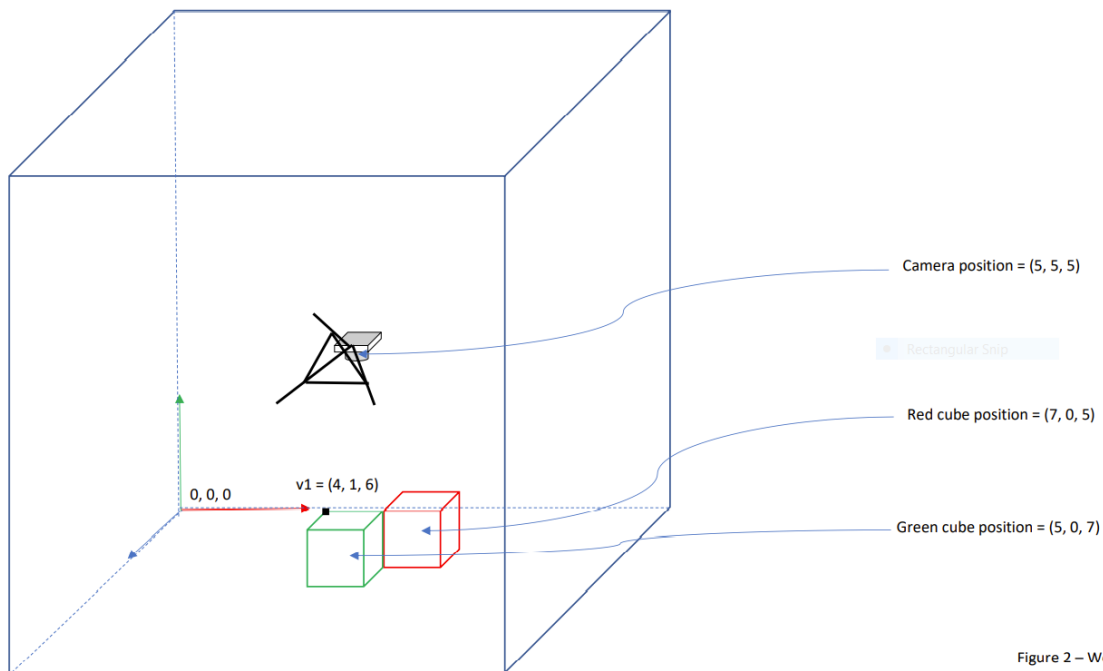
If you run the program, then the following window should appear



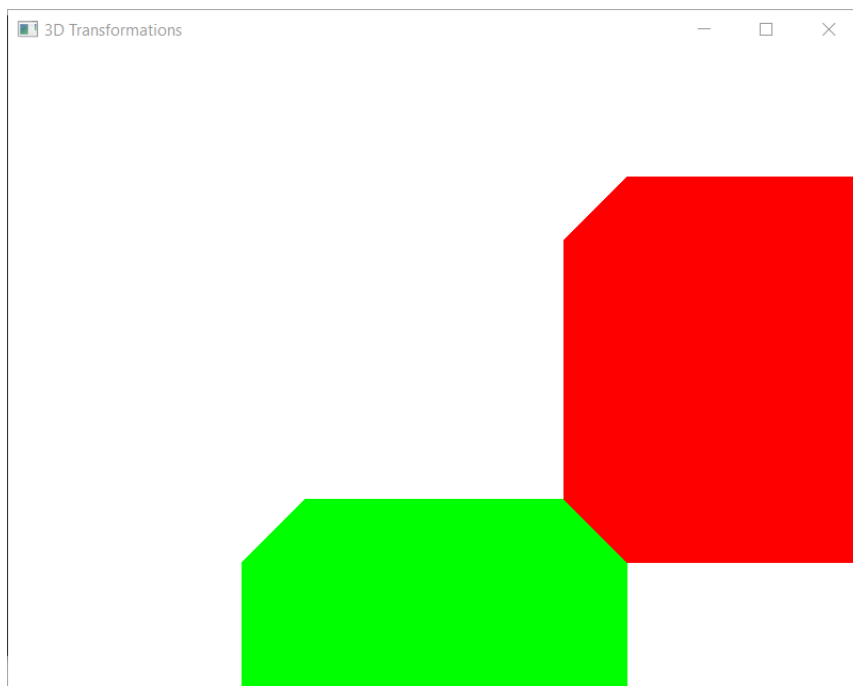
There are 2 cubes (red and green) but they are both in the same location.

Task 1

Set up the cubes and camera so that they are in the same positions as figure 2 in chapter 6 of the lecture notes.



When you run the program, it should look like figure 6 in chapter 6 of the lecture notes, like this



Demo – show your program to a teacher.

Part B Set up

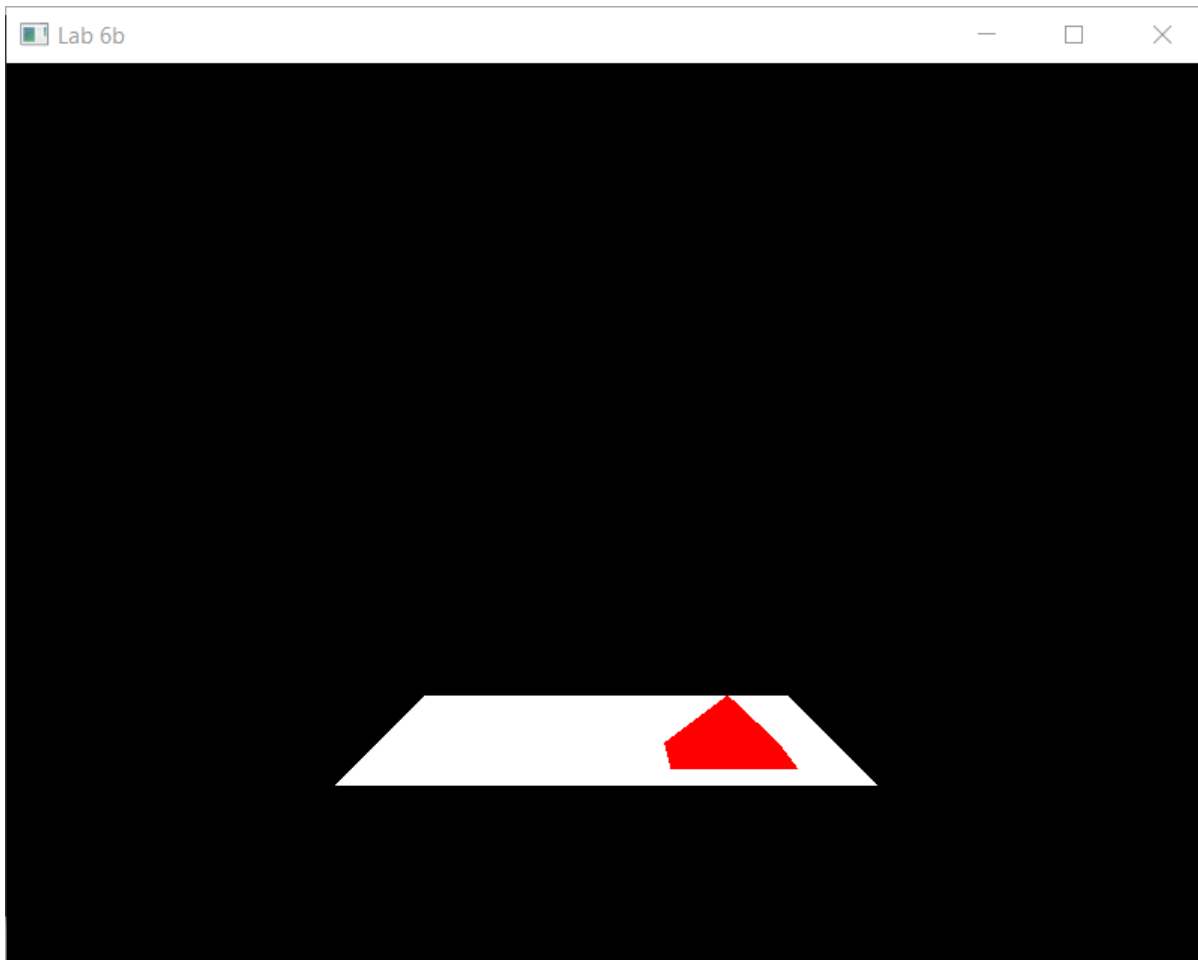
In the COMP3069 directory, create a new project called

Lab6b

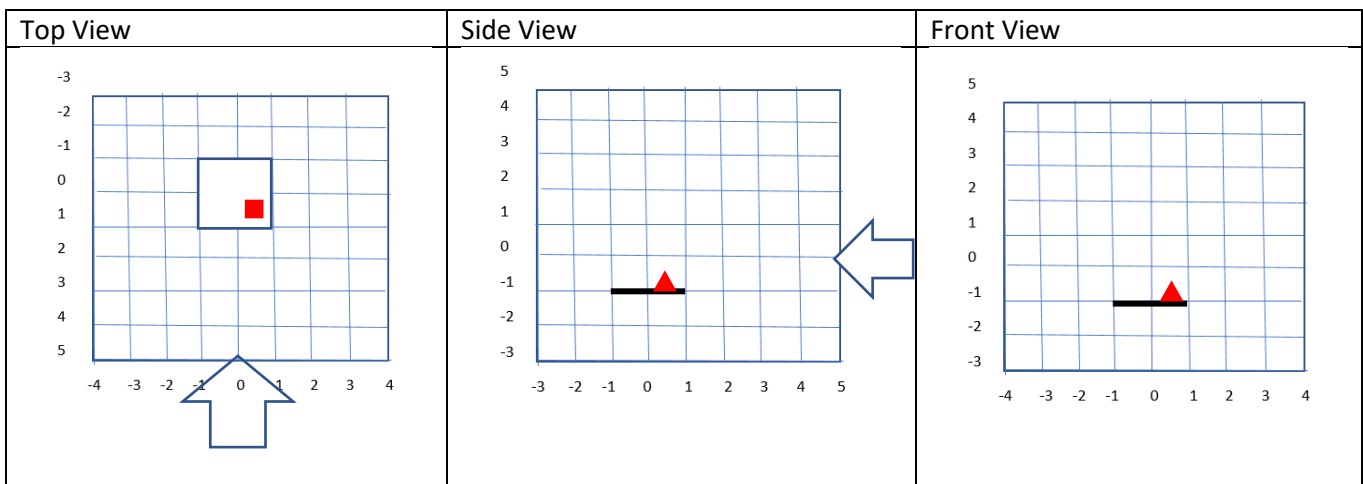
Download Lab6b.cpp from Moodle and copy the code to the Lab6b.cpp in your new project.

Download the vertex shader `mvp1.vert` and fragment shader `uniform1.frag` from Moodle, copy them to your project, and add them to the Source Files in the Solution Explorer.

If you run the program, then the following window should appear

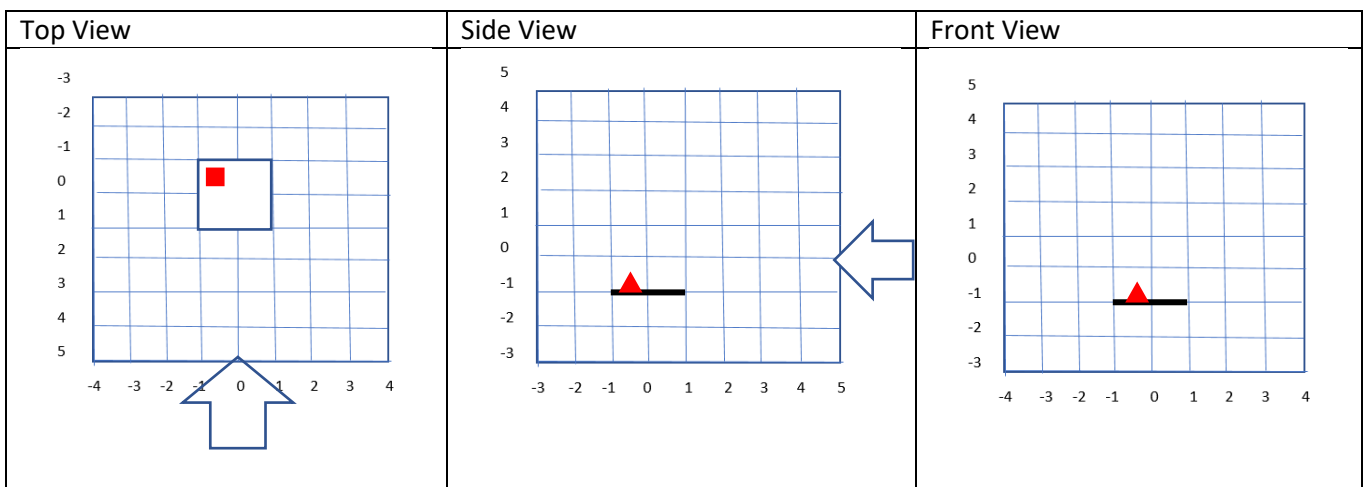
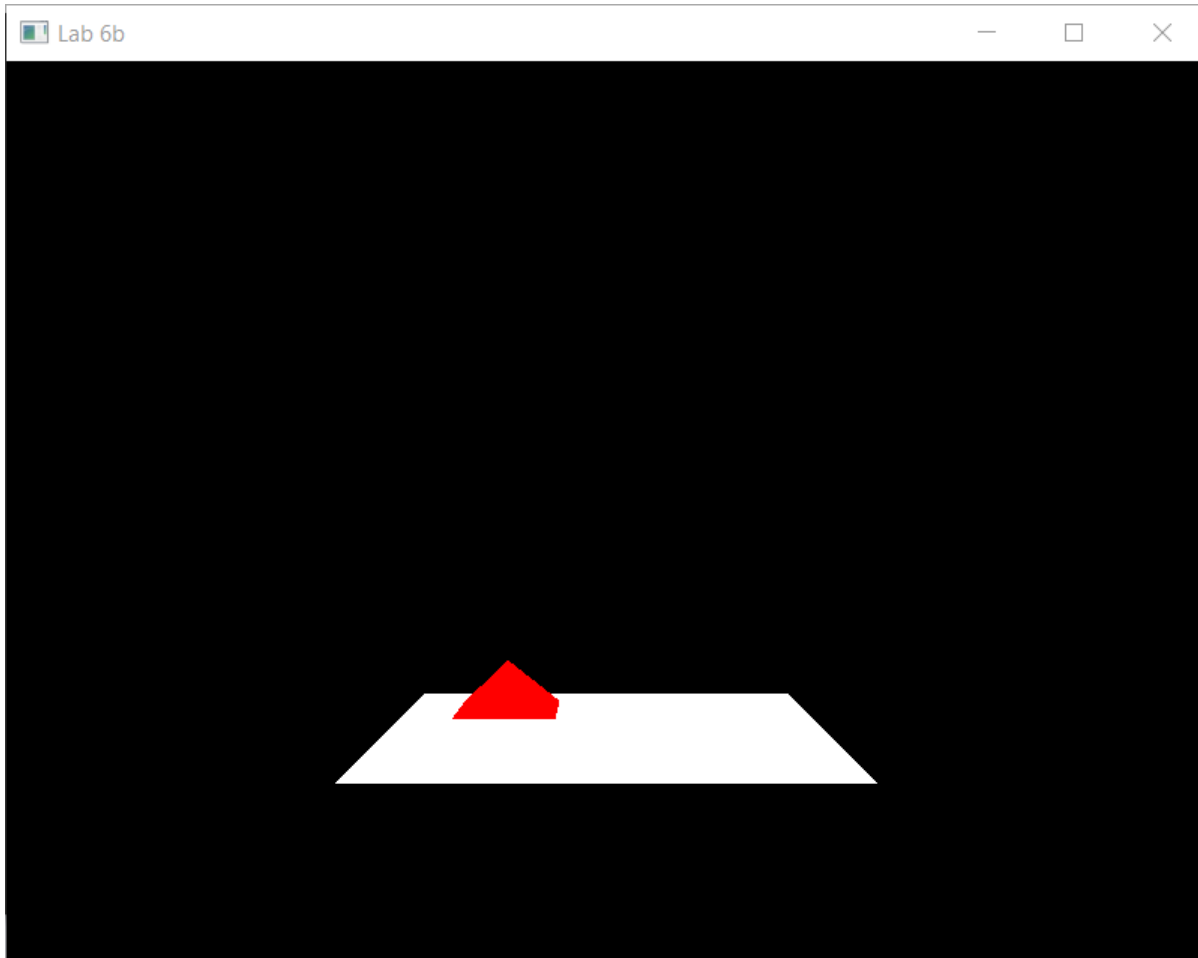


You should see a white plane with a red pyramid in the corner. This is object will be used for this part of the lab.



For the following tasks, the camera is not transformed, it stays at $(0, 0, 5)$. Only the objects are transformed.

Task 2 - Use a transformation to render this image

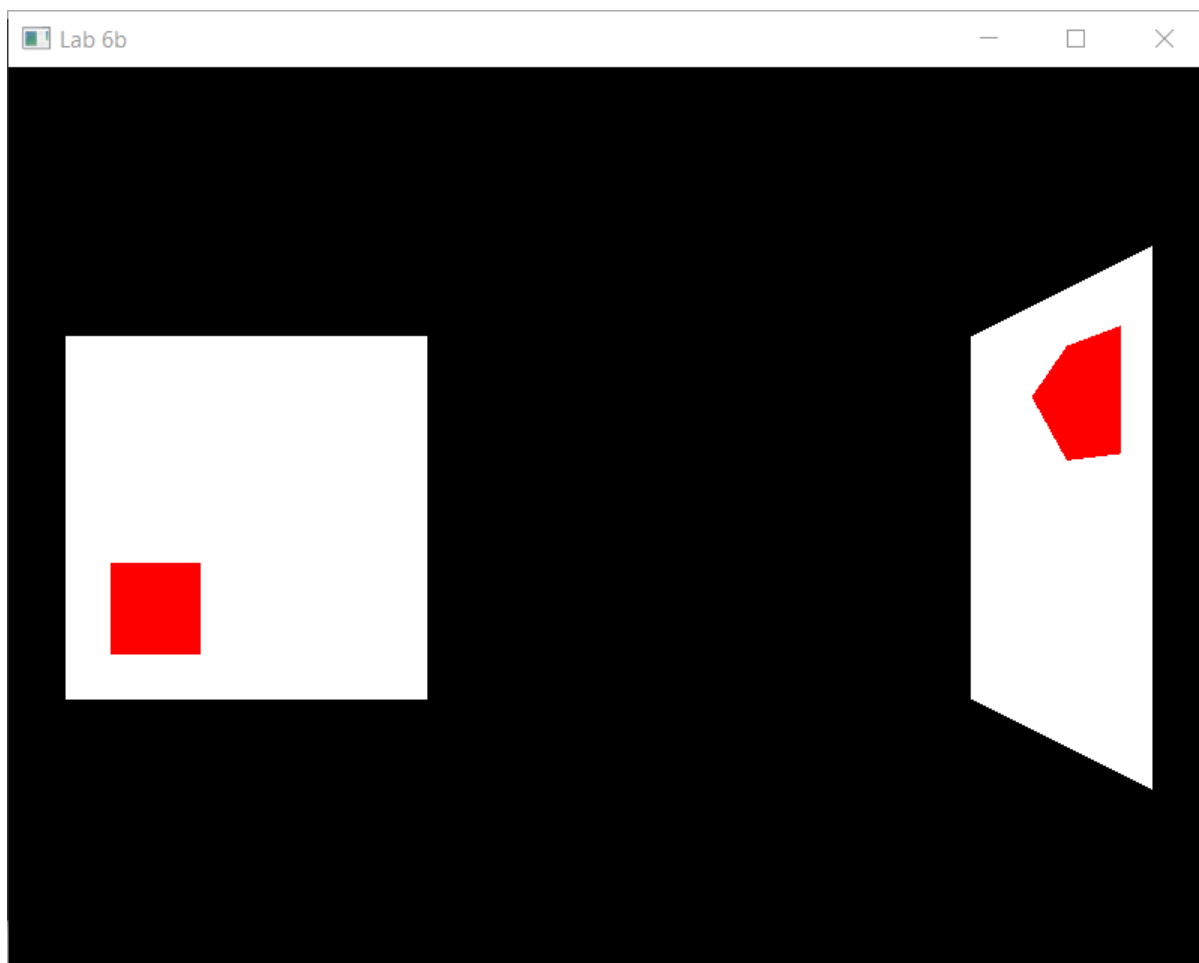


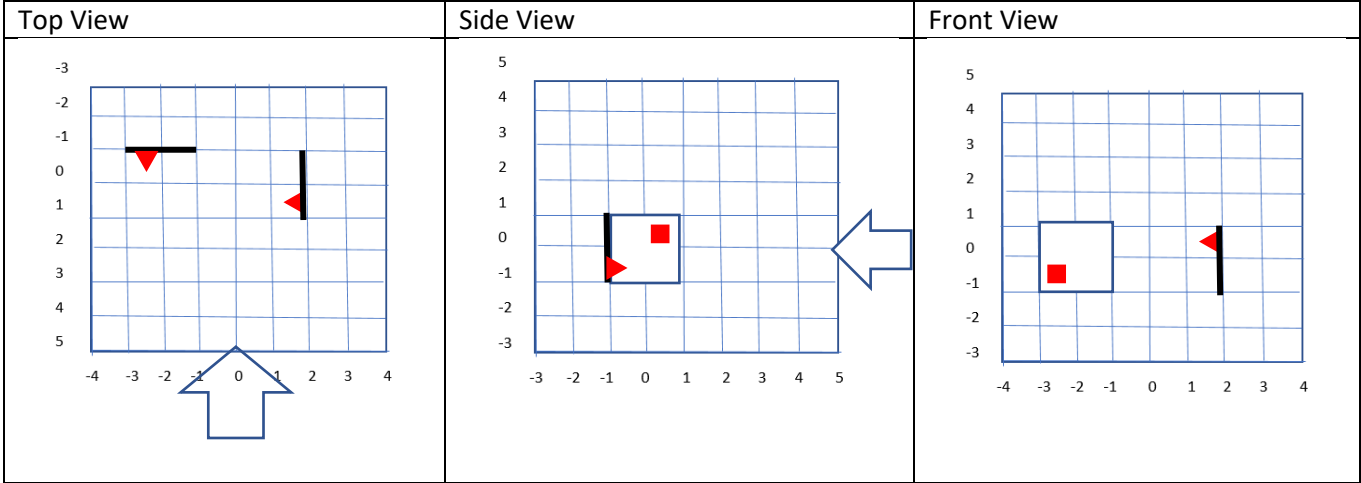
Task 3 - Use a different transformation to the one used in Task 2 to render the same image.

From this point, we need to render multiple copies of the same model, so change the render loop to make it look like

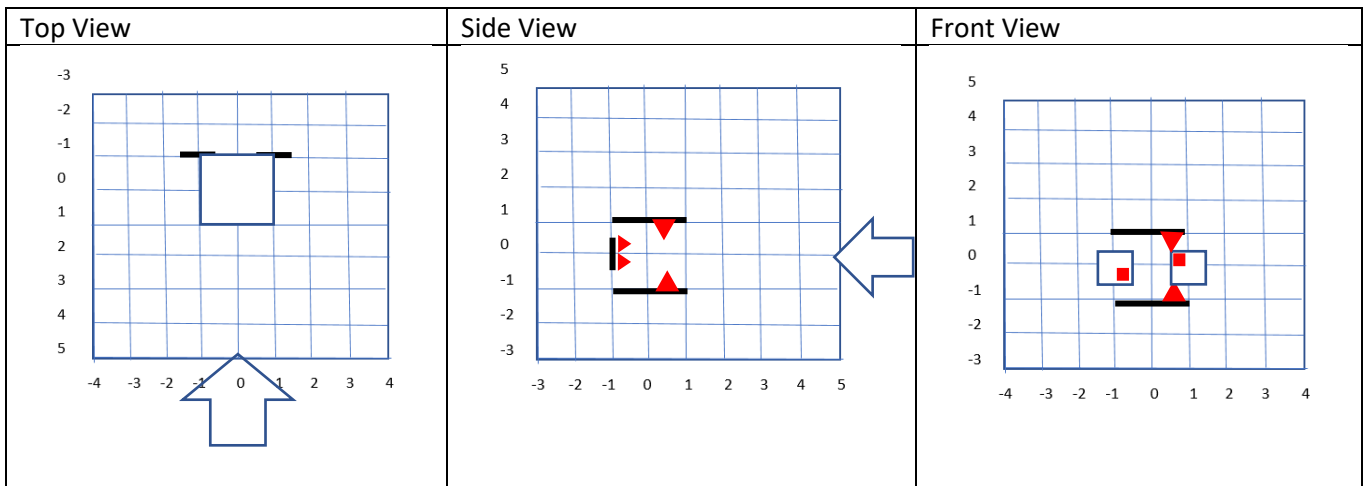
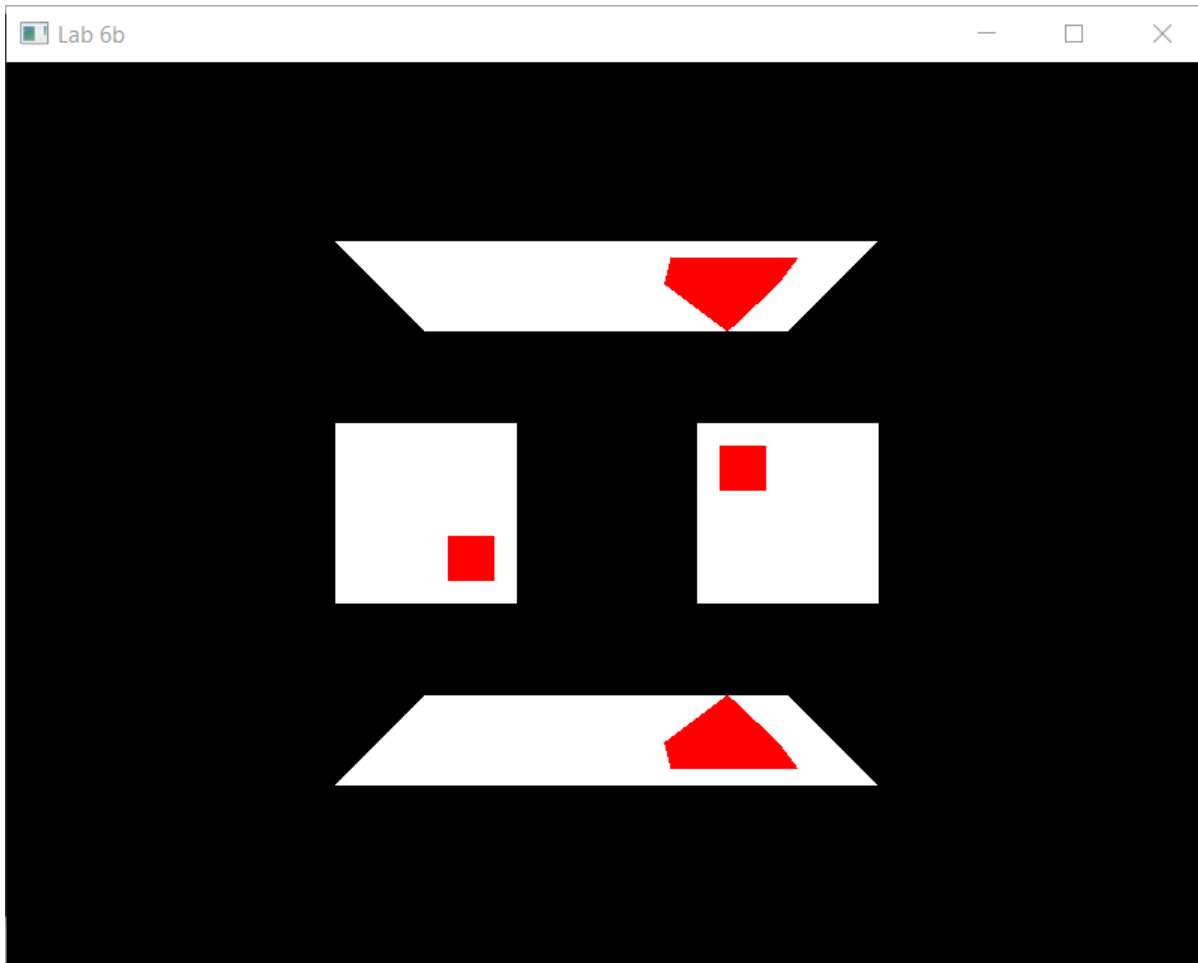
```
157 glm::mat4 view = glm::mat4(1.f);
158 view = glm::translate(view, -glm::vec3(0.0f, 0.0f, 5.0f));
159 int vloc = glGetUniformLocation(shaderProgram, "view");
160 glUniformMatrix4fv(vloc, 1, GL_FALSE, glm::value_ptr(view));
161
162 glm::mat4 projection = glm::mat4(1.f);
163 projection = glm::perspective(glm::radians(45.f), (float)800 / (float)600, .1f, 10.f);
164 int ploc = glGetUniformLocation(shaderProgram, "projection");
165 glUniformMatrix4fv(ploc, 1, GL_FALSE, glm::value_ptr(projection));
166
167 glm::mat4 model = glm::mat4(1.f);
168 int mloc = glGetUniformLocation(shaderProgram, "model");
169 glUniformMatrix4fv(mloc, 1, GL_FALSE, glm::value_ptr(model));
170 glBindVertexArray(VAO);
171 glDrawArrays(GL_TRIANGLES, 0, 18);
172
173 model = glm::mat4(1.f);
174 mloc = glGetUniformLocation(shaderProgram, "model");
175 glUniformMatrix4fv(mloc, 1, GL_FALSE, glm::value_ptr(model));
176 glBindVertexArray(VAO);
177 glDrawArrays(GL_TRIANGLES, 0, 18);
178 glBindVertexArray(0);
```

Task 4 - Use several transformations in sequence on these 2 objects to render this image





Task 5 - Use several transformations in sequence on these 4 objects to render this image



Task 6 – use the cube from part A and/or the shape from part B, with a camera viewing the scene from a specified position and with a given orientation. First imagine what you want to render, then apply the transformations (translate, rotate, scale) to the objects and camera, and try to achieve the desired result. We used multiple models in the same scene in Lab4 2D Transformations. Have a look if you can't remember.

1. Try to use multiple objects, maybe a pile of cubes.
2. Try to make a person out of cubes
3. Try to find an interesting camera angle. You'll probably need a camera rotation which is not a multiple of 90° .
4. Imagine a new 3D shape and add the new model to your scene.

